

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Operating Systems **COURSE CODE:** CSA04

COURSE OUTCOME COVERED

CO4: Optimize various file allocation strategies and disk scheduling algorithms to identify optimal methods for enhancing system performance.

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:100

Annexure A

INDUSTRY PROBLEM SOLVING TASK

Code of Conduct:

I ANISH K/192521288 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

INDUSTRY PROBLEM SOLVING TASK

S.No	Industry Problem Solving Task Questions	Marks
1	A cloud storage company is facing performance issues due to frequent file access delays. Analyze whether contiguous, linked, or indexed allocation would be most suitable for improving access speed and justify your choice with reasons.	10
2	A data center experiences high disk head movement while processing user requests. Suggest the most appropriate disk scheduling algorithm (FCFS, SSTF, SCAN) to reduce seektime and explain how it improves performance in this scenario.	10
3	An operating system uses a bitmap for free space management but struggles with slow allocation when the disk size increases. Propose an optimization technique to improve the efficiency of bitmap-based allocation and justify your solution.	10
4	A real-time application requires consistent and predictable disk access times. Evaluate whether SCAN or C-SCAN would be more suitable and explain your reasoning based on system requirements.	10
5	A company wants to design a hybrid storage system combining file allocation and disk scheduling techniques. Suggest a suitable combination (allocation + scheduling) for handling large-scale data access efficiently and justify your choice based on performance factors.	10
6	Intelligent File Allocation and Storage Optimization System Industry Context: Cloud storage / Data Centers Design a file-storage simulator for a data center that supports Contiguous, Sequential/Linked, and Indexed allocation. Problem Requirements: - Accept file creation requests with file name and required number of blocks. - Implement all three allocation strategies. - Maintain a dynamic disk-block map. - Support file creation, deletion, extension, and block release. - Detect insufficient storage. - Calculate internal and external fragmentation. - Compare storage utilization and access efficiency.	10

	Recommend the most suitable allocation strategy based on file size and access pattern. Advanced Challenge: Develop an algorithm that automatically selects the allocation strategy for each file based on its size, growth rate, and sequential/random access requirements. Expected Outcome: A storage-management dashboard showing block allocation, fragmentation, utilization, and recommended allocation method.	
7	Unix File System Inode and Buffer Cache Simulator Industry Context: Operating-system kernel / Storage systems Develop a simulator that models the internal operation of a Unix-like file system using inodes and buffer cache. Problem Requirements: - Maintain an inode table containing file metadata. - Implement simulated versions of: - ialloc() - ifree() - namei() - alloc() - free() - Implement pathname resolution using namei(). - Maintain a buffer cache for disk blocks. - Simulate cache hit and cache miss situations. - Track dirty and clean buffers. - Implement buffer replacement when the cache is full. - Calculate cache hit ratio and disk-access reduction. Advanced Challenge: Compare different buffer-cache replacement strategies and determine which provides the best performance for different workloads. Expected Outcome: A simulation showing the complete path: Application → System Call → Inode → Buffer Cache → Disk	10

8	Distributed Network File System with Mount/Unmount Management Industry Context: Enterprise servers / Distributed storage Design a Network File System (NFS)-like simulator where multiple client machines access files stored on a remote server. Problem Requirements: • Create a centralized file server. • Simulate multiple clients. • Implement file creation, opening, reading, writing, and deletion. • Implement mounting and unmounting of remote file systems. • Maintain client-side mount tables. • Handle simultaneous file access. • Simulate server failure and recovery. • Detect invalid mount/unmount operations. • Maintain file consistency between clients and server. Advanced Challenge: Design a mechanism to handle concurrent writes to the same file while minimizing conflicts and maintaining consistency. Expected Outcome: A distributed file-system simulator showing: Client → Network → File Server → File System → Disk	10
9	Intelligent File-System Recovery and Crash Consistency System Industry Context: Database servers / Banking / Mission-critical systems Develop a simulator that investigates what happens when a system crashes during file-system operations. Problem Requirements: • Simulate file creation, deletion, allocation, and modification. • Maintain metadata such as inodes and free-block information. • Generate random system crashes during operations. • Identify inconsistent states after a crash. • Detect: ○ Lost blocks ○ Duplicate block allocation ○ Invalid inode references ○ Orphan files	10

	○ Corrupted free-space information • Implement a recovery mechanism. • Restore the file system to a consistent state. Advanced Challenge: Compare different recovery approaches and calculate data loss, recovery time, and consistency level. Expected Outcome: A File System Health and Recovery Tool that reports: Normal State → Crash → Corruption Detection → Recovery → Verified File System	
10	High-Performance I/O and File-System Optimization Simulator Industry Context: Cloud platforms / Video streaming / High-performance computing Develop a simulator that optimizes file-system performance for a workload involving thousands of file operations. Problem Requirements: • Generate read/write requests from multiple applications. • Simulate different file sizes and access patterns. • Implement file allocation and disk-block management. • Simulate the I/O subsystem and request queues. • Include buffer caching. • Measure: ○ File access time ○ Disk I/O operations ○ Cache hit ratio ○ Throughput ○ Storage utilization ○ Fragmentation • Compare sequential and random file access. • Identify system bottlenecks. Advanced Challenge: Build an adaptive optimization algorithm that dynamically changes caching, allocation, and I/O scheduling decisions according to the workload.	10

	Expected Outcome: A performance analyzer that recommends the optimal configuration for workloads such as: • Video streaming • Database transactions • Web-server logs • Large scientific datasets • Cloud file storage	
--	--	--

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding ; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

Q1. A cloud storage company is facing performance issues due to frequent file access delays. Analyze whether contiguous, linked, or indexed allocation would be most suitable for improving access speed and justify your choice with reasons.

Answer

File allocation is the method used by an operating system to allocate disk blocks to files. The three common methods are contiguous allocation, linked allocation, and indexed allocation. For a cloud storage company experiencing frequent file-access delays, contiguous allocation is the most suitable when access speed is the primary requirement.

1. Contiguous Allocation

In contiguous allocation, all blocks of a file are stored in consecutive locations on the disk.

For example, if a file requires five blocks:

File A → [20][21][22][23][24]

The operating system stores the file continuously from block 20 to block 24.

The directory needs to maintain:

Starting Block = 20

Length = 5

2. Linked Allocation

In linked allocation, the blocks can be located anywhere on the disk. Each block contains a pointer to the next block.

[10] → [35] → [18] → [50] → [72]

It is good for file growth, but random access is slower because the system may need to follow several pointers.

3. Indexed Allocation

In indexed allocation, an index block contains the addresses of all blocks belonging to a file.

Index Block			
/			\
10	35	18	50

It supports random access well but requires an additional index block.

Comparison

Feature	Contiguous	Linked	Indexed
Access speed	Very high	Low	High
Random access	Excellent	Poor	Good
Sequential access	Excellent	Good	Good
File growth	Difficult	Easy	Easy
External fragmentation	High	Low	Low
Pointer overhead	None	High	Index-block overhead

Why Contiguous Allocation?

For frequent file access:

Blocks are physically close together.

Disk head movement is reduced.

Sequential reading is very fast.

Random access is also efficient.

Address calculation is simple.

There is no pointer traversal overhead.

Limitation

The main disadvantages are external fragmentation and difficulty in expanding a file if adjacent blocks are unavailable.

Conclusion

Therefore, contiguous allocation is the best choice when high file-access speed is the main requirement. However, for a cloud system with frequent file growth and highly dynamic storage, indexed allocation may be considered as an alternative. The assessment specifically asks for this comparison in the context of improving access speed.

Q2. A data center experiences high disk head movement while processing user requests. Suggest the most appropriate disk scheduling algorithm (FCFS, SSTF, SCAN) to reduce seek time and explain how it improves performance.

Answer

Disk scheduling determines the order in which pending disk I/O requests are serviced. The main objective is to reduce seek time and disk-head movement.

The three algorithms given in the question are:

FCFS
SSTF
SCAN

For the given data-center scenario, SSTF (Shortest Seek Time First) is an appropriate choice for reducing immediate seek time.

1. FCFS

FCFS services requests in the order in which they arrive.

Example:

Request Queue:

98 → 183 → 37 → 122 → 14

The disk head follows the same order.

The problem is that the head may travel a very large distance between consecutive requests.

2. SSTF

SSTF selects the request closest to the current head position.

Suppose:

Current head = 50

Requests = 20, 45, 70, 90, 10

Distances:

50 → 45 = 5

50 → 70 = 20

50 → 20 = 30

50 → 90 = 40

50 → 10 = 40

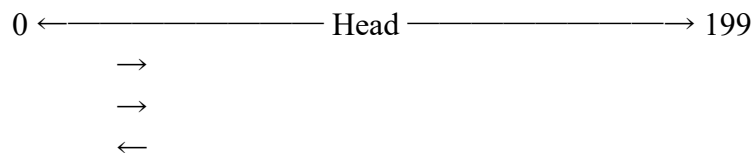
Therefore, SSTF selects:

50 → 45

The next closest request is then selected.

3. SCAN

SCAN moves the disk head in one direction while servicing requests and then reverses direction.



It provides better fairness than SSTF but may travel farther than the nearest-request approach.

Comparison

Algorithm	Seek Time	Fairness	Main Advantage
FCFS	High	High	Simple
SSTF	Low	Moderate	Reduces immediate seek time
SCAN	Low/Moderate	High	Elevator-like servicing

Advantages of SSTF

- Selects the nearest request.
- Reduces unnecessary head movement.
- Reduces average seek time.
- Improves response time.
- Increases disk utilization.
- Suitable for workloads with many pending requests.

Limitation

SSTF can cause starvation. A request located far from the current head may wait for a long time if new requests continuously arrive near the head.

Conclusion

For the stated problem of high disk-head movement and the need to reduce seek time, SSTF is the most direct choice among FCFS, SSTF and SCAN. It services the closest pending request first and therefore generally reduces immediate seek distance.

Q3. An operating system uses a bitmap for free-space management but struggles with slow allocation when the disk size increases. Propose an optimization technique to improve the efficiency of bitmap-based allocation and justify your solution.

Answer

A bitmap is a free-space management technique in which each disk block is represented by one bit.

For example:

0 → Free

1 → Allocated

Bitmap:

1 1 0 0 1 0 1 1

Here, blocks 3, 4 and 6 are free.

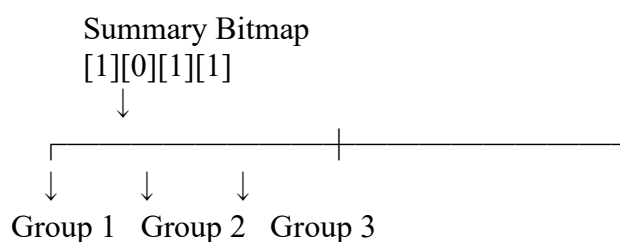
Problem

When the disk becomes very large, the bitmap also becomes large. Searching the entire bitmap to find free blocks can increase allocation time.

Proposed Solution: Hierarchical Bitmap

A suitable optimization is to use a hierarchical bitmap with summary information.

Instead of searching the complete bitmap, divide the disk into groups.



The summary bitmap indicates whether each group contains free blocks.

Working

Suppose:

Group 1 → Contains free block

Group 2 → Completely allocated

Group 3 → Contains free block

Group 4 → Contains free block

The summary bitmap can indicate:

[1][0][1][1]

The system first searches the summary bitmap instead of the complete bitmap.

It then searches only the selected group.

Additional Optimization

Bitwise operations can also be used to identify free bits quickly.

For example:

Bitmap word:
11110000

Free bits:
00001111

The system can process multiple bits simultaneously rather than checking each bit individually.

Advantages

- Reduces bitmap search time.
- Avoids scanning the complete disk.
- Improves block allocation speed.
- Reduces CPU overhead.
- Works effectively with large disks.
- Makes free-space searching more scalable.

Example

Without optimization:

Search:

Block 1 → Block 2 → Block 3 → ... → Block 1,000,000

With hierarchical bitmap:

Search Summary



Find Group 25



Search Group 25



Find Free Block

Conclusion

A hierarchical bitmap with summary information is an effective solution for large disks because it reduces the amount of bitmap data that must be scanned during allocation. This directly addresses the document's requirement to improve bitmap-based allocation efficiency as disk size increases.

Q4. A real-time application requires consistent and predictable disk access times. Evaluate whether SCAN or C-SCAN would be more suitable and explain your reasoning based on system requirements.

Answer

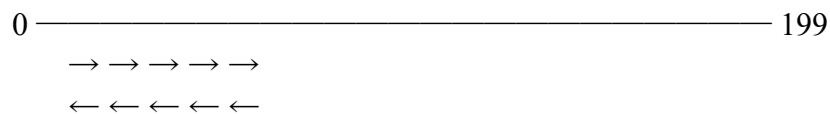
SCAN and C-SCAN are disk scheduling algorithms designed to reduce unnecessary head movement and provide better performance than simple FCFS.

For a real-time application requiring consistent and predictable disk access times, C-SCAN is more suitable.

SCAN

SCAN works similar to an elevator.

The disk head moves in one direction and services all requests along its path. After reaching the end, it reverses direction.

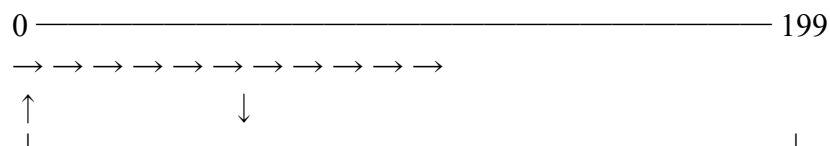


Therefore, requests are serviced in both directions.

C-SCAN

C-SCAN stands for Circular SCAN.

The head services requests in only one direction. When it reaches the end, it returns to the beginning and starts servicing requests again in the same direction.



The return movement is generally not used to service requests.

Comparison

Feature	SCAN	C-SCAN
Service direction	Both directions	One direction
Waiting time	Less uniform	More uniform
Predictability	Good	Better
Fairness	Good	High
Suitable for predictable access	Good	Very suitable

Why C-SCAN?

- Requests are serviced in a uniform direction.
- Waiting time is more predictable.
- It provides more uniform service to different disk locations.
- It reduces the possibility of requests receiving significantly different waiting times.
- It is suitable for systems where consistency is more important than servicing the absolute nearest request.

Example

If requests are distributed across the disk, SCAN may service some requests quickly depending on the direction of the head.

C-SCAN provides a more uniform waiting pattern because every request waits for the head to make its circular sweep.

Conclusion

For the given real-time requirement of consistent and predictable disk access, C-SCAN is preferable to SCAN because it provides more uniform waiting time and predictable service patterns. This matches the requirement stated in the assessment.

Q5. A company wants to design a hybrid storage system combining file allocation and disk scheduling techniques. Suggest a suitable combination (allocation + scheduling) for handling large-scale data access efficiently and justify your choice based on performance factors.

Answer

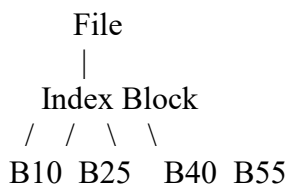
A hybrid storage system should select both an appropriate file allocation technique and a suitable disk scheduling algorithm.

For large-scale data access, a suitable combination is:

Indexed Allocation + C-SCAN

1. Indexed Allocation

In indexed allocation, an index block stores the addresses of the data blocks belonging to a file.



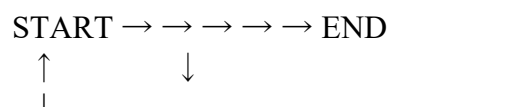
The file's blocks do not need to be contiguous.

Advantages

- Supports random access.
- Supports file growth.
- Does not require contiguous free space.
- Suitable for large files.
- Avoids external-fragmentation problems associated with contiguous allocation.

2. C-SCAN

C-SCAN services disk requests in one direction and then returns to the beginning.



Advantages

- Provides relatively uniform waiting time.
- Reduces unfairness.
- Provides predictable request servicing.
- Suitable for large request queues.

Combined Architecture
Application



Indexed File Allocation



I/O Request Queue



C-SCAN Scheduler



Disk

Why This Combination?

Requirement	Indexed + C-SCAN
Large files	Excellent
Random access	Good
File growth	Good
Large disk	Suitable
Predictable scheduling	Good
External fragmentation	Low
Scalability	High

Example

Suppose a large database file requires 1,000 blocks.

Contiguous allocation may require 1,000 consecutive free blocks, which may not be available.

Indexed allocation can place the blocks at different locations:

Index



10, 25, 80, 101, 250, 500, ...

C-SCAN then manages the disk requests in an organized circular sequence.

Conclusion

Indexed allocation + C-SCAN provides a good balance between flexible large-file storage and predictable disk scheduling. It is therefore suitable for a large-scale storage system where both file-management flexibility and disk-I/O performance are important. The assessment specifically asks for a combination based on large-scale data-access performance.

Q6. Intelligent File Allocation and Storage Optimization System

The task requires support for contiguous, sequential/linked and indexed allocation, dynamic disk-block management, file creation/deletion/extension, fragmentation calculation, storage utilization and automatic recommendation.

Answer

Objective

The objective is to design an Intelligent File Allocation and Storage Optimization System that automatically chooses the most suitable allocation strategy based on file characteristics.

Inputs

The system accepts:

File name
Number of required blocks
File access pattern
File growth rate

Example:

File Name : video.mp4
Required Blocks: 100
Access Pattern : Sequential
Growth Rate : Low
Allocation Methods
A. Contiguous Allocation
[10][11][12][13][14]

All blocks are consecutive.

Suitable for: sequential access and high-speed access.

B. Linked Allocation
[10] → [35] → [72] → [90]

Blocks may be located anywhere.

Suitable for: files that grow frequently.

C. Indexed Allocation

Index Block



10 → 35 → 72 → 90

Suitable for: large files and random access.

Intelligent Selection Algorithm

START



Read file information



Check file size



Check access pattern



Check growth rate



Select allocation strategy



Check available blocks



Allocate blocks



Update disk map



Calculate fragmentation



Calculate utilization



Display recommendation



STOP

Selection Logic

IF sequential access AND low growth

→ Contiguous

ELSE IF high growth rate

→ Linked

ELSE IF random access OR large file

→ Indexed

Operations

The simulator must support:

$$\frac{\begin{array}{l} \text{File creation} \\ \text{File deletion} \\ \text{File extension} \\ \text{Block allocation} \\ \text{Block release} \\ \text{Insufficient-storage detection} \\ \text{Performance Metrics} \\ \text{Storage Utilization} \\ \text{Used Blocks} \end{array}}{\text{Total Blocks} \\ \text{Fragmentation}} \times 100$$

The simulator calculates internal and external fragmentation depending on the allocation strategy and storage state.

Example Output

File Name : database.db
 Size : 500 blocks
 Access Pattern : Random
 Growth Rate : Medium

Recommended Method : INDEXED

Storage Utilization : 85%
 External Fragmentation : 4%
 Access Efficiency : High

Conclusion

The proposed system provides intelligent allocation by considering file size, growth rate and access pattern. It can improve storage utilization and access efficiency while providing a dashboard for visualization. The document's expected outcome is specifically a storage-management dashboard showing allocation, fragmentation, utilization and recommendation.

Q7. Unix File System Inode and Buffer Cache Simulator

The document requires an inode table, simulated `ialloc()`, `ifree()`, `namei()`, `alloc()`, `free()`, pathname resolution, buffer cache, hit/miss detection, dirty/clean buffers and replacement.

Answer

Objective

The objective is to simulate how a Unix-like operating system manages files using inodes and buffer cache.

Overall Operation
Application



System Call



Inode



Buffer Cache



Disk

1. Inode

An inode stores file metadata such as:

File size

File type

Permissions

Owner

Block addresses

Other file information

Example:

Inode 25

File: report.txt

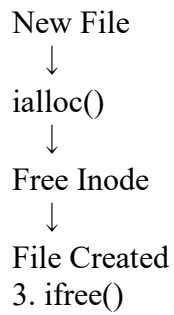
Size: 4096 bytes

Type: Regular

Blocks: 10, 20, 30

2. ialloc()

ialloc() allocates a free inode when a new file is created.



3. ifree()

Releases an inode after a file is deleted.

4. alloc()

Allocates a free disk block.

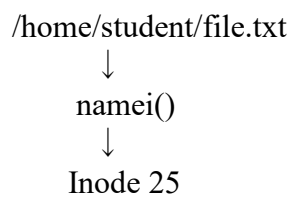
5. free()

Releases a previously allocated block.

6. namei()

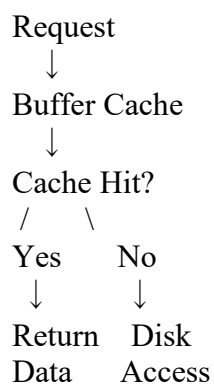
namei() performs pathname resolution.

Example:



7. Buffer Cache

The buffer cache stores frequently accessed disk blocks in memory.





Update Cache

Cache Hit Ratio

Cache Hit Ratio =

Cache Hits / Total Requests $\times 100$

If:

Hits = 80

Requests = 100

then:

Hit Ratio = 80%

Dirty and Clean Buffers

Clean buffer: Data matches the disk.

Dirty buffer: Data has been modified in memory but has not yet been written to disk.

Buffer Replacement

When the cache becomes full, a replacement strategy can be used, such as:

FIFO

LRU

Clock

Conclusion

The simulator demonstrates the complete path:

Application $\rightarrow$ System Call $\rightarrow$ Inode $\rightarrow$ Buffer Cache $\rightarrow$ Disk

and measures cache hit ratio and disk-access reduction. This directly matches the expected outcome specified in the assessment.

Q8. Distributed Network File System with Mount/Unmount Management

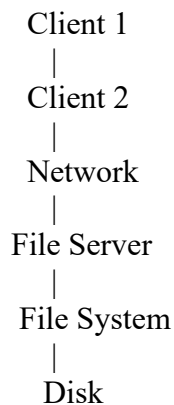
The question requires a centralized file server, multiple clients, file operations, mounting/unmounting, mount tables, simultaneous access, server failure/recovery and consistency.

Answer

Objective

The objective is to design an NFS-like distributed file-system simulator where multiple client computers access files stored on a remote server.

Architecture



1. Centralized File Server

The server stores the actual files and manages access to them.

2. Multiple Clients

Clients send requests to the server through the network.

Client → Network → Server → File System → Disk

3. File Operations

The simulator supports:

Create
Open
Read
Write
Delete

4. Mounting

Mounting makes the remote file system available through a directory on the client.

Example:

Remote:

/server/shared

Client mount point:

/mnt/shared

After mounting:

Client



/mnt/shared



Network



Server

5. Unmounting

Unmounting disconnects the remote file system from the client mount point.

6. Mount Table

Each client maintains a mount table.

Client	Remote FS	Mount Point
--------	-----------	-------------

C1	/server/data	/mnt/data
----	--------------	-----------

C2	/server/data	/mnt/data
----	--------------	-----------

7. Invalid Operations

The system should detect:

- Mounting an already mounted file system
- Unmounting a non-mounted file system
- Invalid mount point
- Access when server is unavailable

8. Simultaneous Access

If two clients write to the same file, the simulator can use file locking.

Client 1 → LOCK → WRITE → UNLOCK

Client 2 → WAIT

Client 2 → LOCK → WRITE → UNLOCK

This prevents conflicting updates.

9. Server Failure and Recovery

Normal Operation



Server Failure



Requests Fail



Server Recovery



Clients Reconnect



Consistency Check

Conclusion

The simulator demonstrates:

Client → Network → File Server → File System → Disk

and provides mounting, unmounting, concurrent access, failure recovery and consistency management, exactly matching the expected distributed-file-system functionality in the assessment.

Q9. Intelligent File-System Recovery and Crash Consistency System

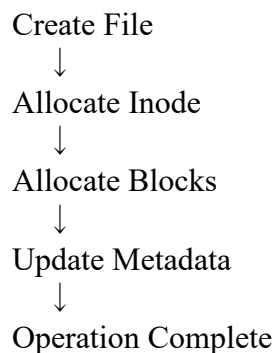
The document requires simulation of crashes during file-system operations and detection/recovery of lost blocks, duplicate allocations, invalid inode references, orphan files and corrupted free-space information.

Answer Objective

The objective is to design a system that detects file-system corruption after a crash and restores the file system to a consistent state.

Normal File Operation

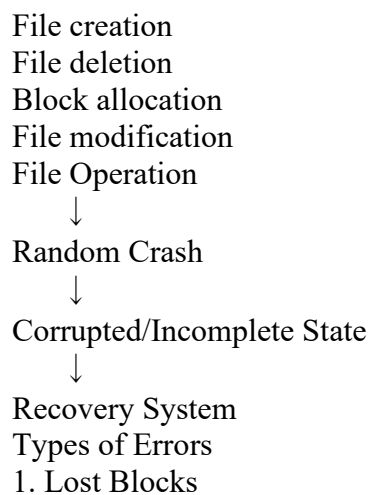
For example, during file creation:



If a crash occurs between these operations, the file system may become inconsistent.

Crash Simulation

The system randomly generates crashes during:



A block is marked as allocated but no file references it.

Block 50 → Allocated

No file → Block 50

2. Duplicate Allocation

Two files reference the same block.

File A → Block 60

File B → Block 60

3. Invalid Inode Reference

An inode points to an invalid or unavailable block.

4. Orphan File

A file exists but is not correctly connected to the directory structure.

5. Corrupted Free-Space Information

The free-space structure does not correctly represent which blocks are free or allocated.

Recovery Process

START



Detect Crash



Check Inodes



Check Block Allocation



Find Lost Blocks



Find Duplicate Blocks



Check Inode References



Find Orphan Files



Repair Metadata



Update Free-Space Map



Verify File System



CONSISTENT STATE

Example Output
Crash Detected

Lost Blocks : 3
Duplicate Blocks : 1
Orphan Files : 2
Invalid Inodes : 1

Recovery Started...
Recovery Completed

File System Status: CONSISTENT
Performance Evaluation

The system can compare recovery techniques using:

1. Data loss
2. Recovery time
3. Consistency level

Conclusion

The proposed recovery system detects file-system inconsistencies, repairs metadata and restores the file system to a consistent condition. The expected outcome in the document is:

Normal State → Crash → Corruption Detection → Recovery → Verified File System.

Q10. High-Performance I/O and File-System Optimization Simulator

The document requires the system to generate read/write requests, simulate file allocation and I/O queues, include buffer caching and measure access time, I/O operations, cache hit ratio, throughput, utilization and fragmentation.

Answer

Objective

The objective is to develop a High-Performance I/O and File-System Optimization Simulator that analyzes thousands of file operations and recommends the best configuration for different workloads.

System Architecture

Applications



Read/Write Requests



File System



File Allocation



I/O Request Queue



Disk Scheduling



Buffer Cache



Disk

1. Request Generation

Multiple applications generate:

READ
WRITE
READ
READ
WRITE
...

The system simulates different file sizes and access patterns.

2. File Allocation

The simulator can use:

Contiguous allocation

Linked allocation

Indexed allocation

3. I/O Scheduling

The system manages the request queue using appropriate disk scheduling algorithms.

4. Buffer Cache

Frequently accessed blocks are kept in memory to reduce disk access.

Performance Metrics

A. File Access Time

Measures how long it takes to access a requested file/block.

B. Disk I/O Operations

Measures the number of disk read/write operations.

C. Cache Hit Ratio

Cache Hit Ratio =

$\text{Cache Hits} / \text{Total Requests} \times 100$

D. Throughput

Throughput =

$\text{Data Processed} / \text{Time}$

E. Storage Utilization

Storage Utilization =

$\text{Used Storage} / \text{Total Storage} \times 100$

F. Fragmentation

Measures inefficient use of storage caused by block distribution.

Workload-Based Optimization

Video Streaming

Usually involves large sequential reads.

Allocation $\rightarrow$ Contiguous

Scheduling $\rightarrow$ SCAN

Caching $\rightarrow$ Sequential caching

Database Transactions

Usually involve many random accesses.

Allocation → Indexed

Scheduling → SSTF

Caching → Frequently accessed blocks

Web-Server Logs

Usually involve continuous sequential writes.

Allocation → Contiguous/Linked

Scheduling → SCAN

Adaptive Optimization

The advanced challenge requires dynamically changing caching, allocation and I/O scheduling decisions according to workload.

Workload



Monitor Performance



Analyze Access Pattern



Choose Allocation



Choose Scheduling



Choose Cache Strategy



Measure Performance



Optimize Again

Sample Output

Workload : Video Streaming

Average Access Time : 12 ms

Disk I/O Operations : 1250

Cache Hit Ratio : 91%

Throughput : 850 MB/s

Storage Utilization : 87%

Fragmentation : 4%

Recommended:

Allocation → Contiguous

Scheduling → SCAN

Caching → Sequential

Conclusion

The simulator analyzes file-system and I/O performance and recommends an optimal configuration based on workload characteristics. The expected outcome specified in the document is a performance analyzer for workloads including video streaming, database transactions, web-server logs, large scientific datasets and cloud file storage.